ENCONTRO 2 DE 6

Registradores e o formato da instrução

Curso Introdutório de Programação de Baixo Nível



32 bits

Liga ProtoFPGA · UFSC Araranguá

Semana passada, este número ficou pela metade

```
add  $t2, $t0, $t1      →      0x01095020
```

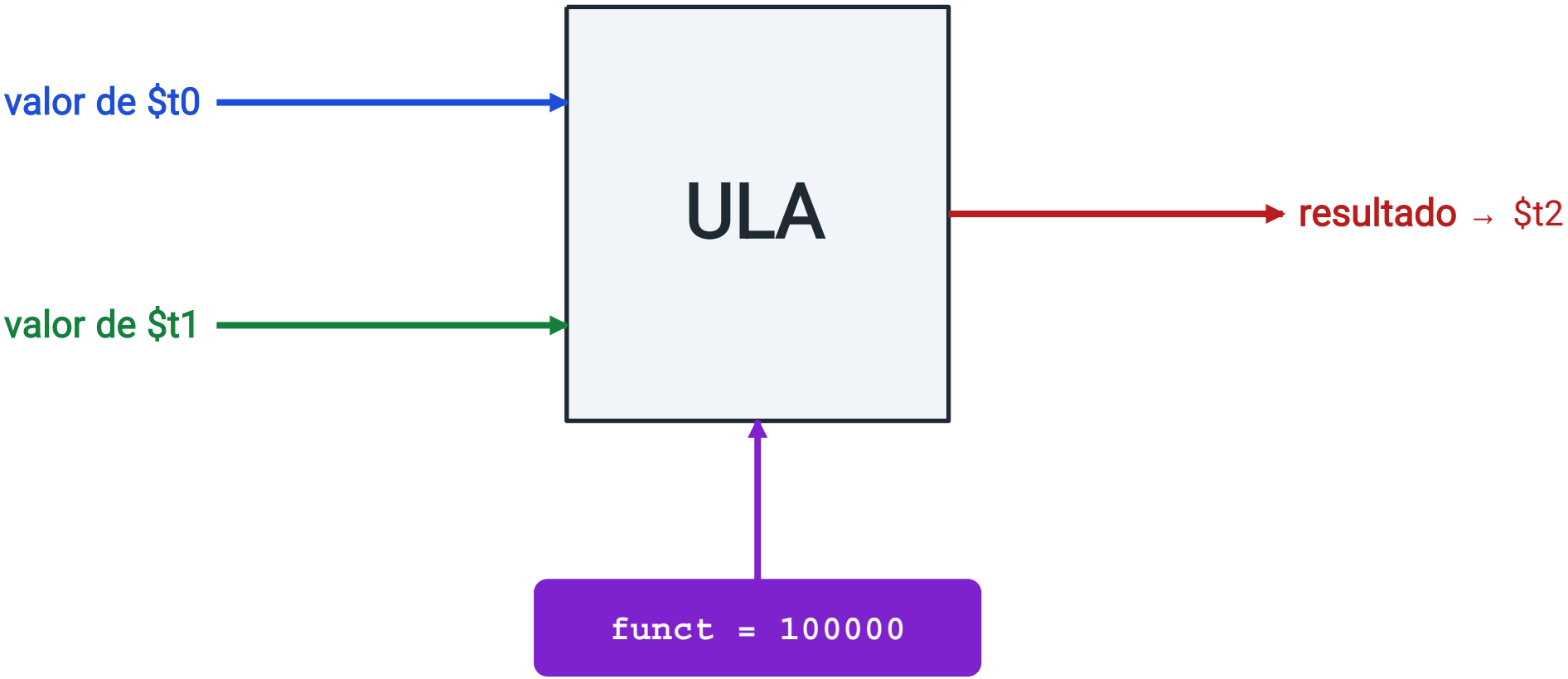


Três perguntas ficaram no ar:

- O que são os blocos com ponto de interrogação?
- Por que cada registrador ocupa exatamente 5 bits?
- E, afinal, o que é um registrador?

Hoje: nenhum ponto de interrogação sobra.

Vocês montam a ULA. Hoje: quem aperta os botões dela



Os bits de controle decidem a operação: somar, subtrair, E, OU…

Bancada e almoxarifado

Registradores — a bancada

- 32, e só 32
- Dentro do processador, ao lado da ULA
- Cada um guarda exatamente 32 bits
- Nome: 5 bits

Memória — o almoxarifado

- Bilhões de posições
- Fora do processador
- Muitas vezes mais lenta
- Endereço: 32 bits

O motivo que costuma passar despercebido: uma soma que nomeasse três posições de memória precisaria de 3 × 32 = 96 bits só de endereços. A instrução inteira tem 32. Contas entre registradores não são só rápidas: são as que cabem.

Iguais no circuito, diferentes na convenção

Número	Nome	Uso combinado
0	`$zero`	Vale sempre 0
1	`$at`	Reservado ao montador
2–3	`$v0–$v1`	Resultados; $v0 escolhe o syscall
4–7	`$a0–$a3`	Argumentos de função
8–15	`$t0–$t7`	Temporários
16–23	`$s0–$s7`	Salvos entre chamadas de função
24–25	`$t8–$t9`	Mais temporários
26–27	`$k0–$k1`	Do sistema operacional — não use
28	`$gp`	Ponteiro global
29	`$sp`	Ponteiro de pilha
30	`$fp`	Ponteiro de quadro
31	`$ra`	Endereço de retorno

Para o hardware, $t0 e $s0 são idênticos.
Os nomes e usos são um acordo entre programadores — e entre programas escritos por pessoas diferentes.

A exceção: $zero.
Está ligado ao zero no circuito.
Escrever nele não tem efeito nenhum — vocês vão conferir.

Fonte: Patterson & Hennessy, Apêndice A, Figura A.6.1

O tamanho do campo decide o tamanho do banco

5 bits

$2^5 = 32$ combinações

```
00000   →  $0    ($zero)
00001   →  $1    ($at)
  ...
01000   →  $8    ($t0)
  ...
11111   →  $31   ($ra)
```

Bits por registrador	Registradores	Bits para 3 registradores
4	16	12
5	32	15
6	64	18

Mais registradores: menos idas à memória, mas cada instrução gasta mais bits só para nomeá-los — e o banco fica maior.
Menos registradores: sobra espaço na instrução, mas o programa vive indo ao almoxarifado.

A instrução R: seis campos, 32 bits



6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
bits 31–26	bits 25–21	bits 20–16	bits 15–11	bits 10–6	bits 5–0
qual família	1ª origem	2ª origem	destino	deslocamento	qual conta

`6 + 5 + 5 + 5 + 5 + 6 = 32`

R de registrador: as três pontas da conta são registradores. Em todas as instruções R, op = 000000. shamt só é usado em deslocamentos (sll, srl); nas demais, vale 00000.

0x01095020, campo por campo

hexadecimal	0	1	0	9	5	0	2	0
binário	0000	0001	0000	1001	0101	0000	0010	0000

reagrupando em 6 · 5 · 5 · 5 · 5 · 6:

op	rs	rt	rd	shamt	funct
000000	01000	01001	01010	00000	100000
0	8	9	10	0	32
formato R	$t0	$t1	$t2	não usado	add

Cada dígito hexadecimal vale exatamente 4 bits — mas os campos não respeitam essa divisão. Por isso se passa pelo binário.

No texto, o destino vem primeiro. Nos bits, por último.

o que você escreve



o que o montador grava



Por que assim? O texto foi feito para ser lido como atribuição — destino ← origens, igual a soma = a + b em C. Os bits foram feitos para o circuito: rs e rt ficam sempre no mesmo lugar, em todos os formatos, e o banco de registradores pode começar a ser lido antes mesmo de se saber qual é a instrução.

op diz a família. funct diz a conta.

Instrução	op	funct	hex	A ULA faz
add	000000	**100000**	0x20	soma
sub	000000	**100010**	0x22	subtração
and	000000	**100100**	0x24	E, bit a bit
or	000000	**100101**	0x25	OU, bit a bit
slt	000000	**101010**	0x2A	compara (encontro 4)

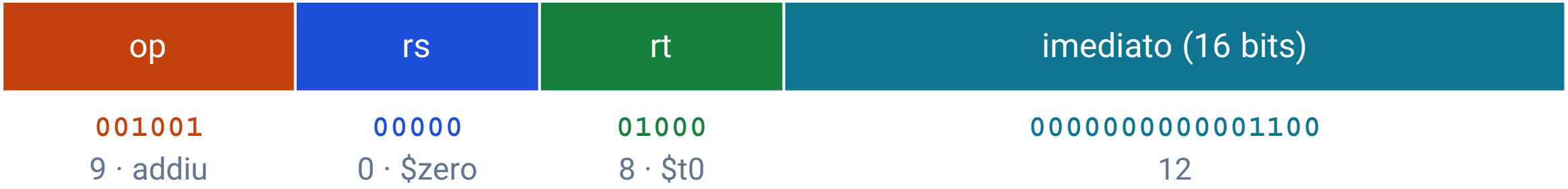
```
      1100   (12)
  E   0101   ( 5)
  =   0100   ( 4)

      1100   (12)
 OU   0101   ( 5)
  =   1101   (13)
```

and e or são as portas lógicas da disciplina, aplicadas 32 vezes lado a lado — uma por bit.

Formato I: um número dentro da instrução

`li $t0, 12` → `addiu $8, $0, 12` → **`0x2408000C`**



Aqui não existe rd. Com só dois registradores, rt passa a ser o destino. O op muda de instrução para instrução: não há funct.

addi 001000	**addiu** 001001	**ori** 001101	**lui** 001111

Trocar 12 por 20: por que só o final mudou?

0x2408000C



0x24080014



Operação, origem e destino ficaram iguais. Só o campo que carrega o valor mudou. Era isto que a resposta do exercício 8 dizia sem poder explicar.

op vem primeiro: ele diz como cortar o resto



add, sub, and, or, slt



addi, addiu, ori, lui · lw, sw · beq



j, jal — encontros 4 e 5

op ocupa sempre os mesmos 6 bits. rs e rt ocupam o mesmo lugar em R e em I.

Complemento de dois, como ferramenta

4 bits	valor
0111	7
0110	6
...	
0001	1
0000	0
1111	−1
1110	−2
...	
1000	−8

Bit da esquerda = 1 → negativo.
Para obter −x: inverta todos os bits e some 1.

```
  1 = 0000 0000 0000 0001
inv = 1111 1111 1111 1110
 +1 = 1111 1111 1111 1111   = -1   (0xFFFF)
```

`addi $s0, $t2, -1` guarda 0xFFFF no imediato. Antes da soma, o processador copia o bit de sinal para completar 32 bits: **`0xFFFF → 0xFFFFFFFF`** (extensão de sinal).

Faixas: 16 bits → −32 768 a 32 767 · 32 bits → −2 147 483 648 a 2 147 483 647

Decomponham comigo

```
sub  $t3, $t0, $t1      →      0x01095822
```

binário: _____ _____ _____ _____



Comparado com o add (0x01095020), quais campos mudaram?
nenhum · só o funct · funct e rd · todos

Atividade

1	Observar — montar o aula2_campos.asm e achar os números dos registradores	5 min
2	Decompor — abrir no papel um código R e um código I	12 min
3	Montar **à mão** — escrever o código de uma instrução e conferir no MARS	10 min
4	Provocar — escrever em $zero, um número grande e uma soma que transborda	8 min

O enunciado está na folha de atividade. Todas as tabelas estão na folha de consulta, na última página.

Duas surpresas

`li $zero, 7`

```
addiu $0, $0, 7

$zero = 0   (continua)
```

O montador aceita. O circuito ignora: $zero está ligado ao zero.

`li $s1, 100000`

```
lui $1, 1           # $at = 65536
ori $17, $1, 0x86a0 # $s1 = 100000
```

100000 não cabe em 16 bits. O montador monta o número em duas metades e precisa de um rascunho: **`$1, o $at`** — por isso ele é reservado.

Confusão 7 do encontro 1, confirmada: uma linha de assembly virou duas instruções de máquina, em dois endereços.

Quando a conta não cabe em 32 bits

```
  0x7FFFFFFF   =   2 147 483 647
+ 0x00000001   =               1
= 0x80000000   =  -2 147 483 648
```

`addu $t2, $t0, $t1`

Aceita o resultado e segue.
`$t2 = -2 147 483 648`

`add  $t3, $t0, $t1`

Detecta o transbordo e para.
`MARS: arithmetic overflow`

Os bits estão certos; o significado não: dois positivos deram negativo. Era esta a diferença entre addi e addiu no encontro 1.

O que fica de hoje

1. Registradores ficam dentro do processador. São 32 porque cada nome ocupa 5 bits.
2. Os nomes $t, $s, $a… são convenção. Só $zero é especial no circuito.
3. Formato R: op · rs · rt · rd · shamt · funct. O destino vai por último nos bits.
4. Formato I: op · rs · rt · imediato. rt é o destino.
5. Números negativos em complemento de dois; somas podem transbordar.

Ninguém decora os códigos. Decora-se a régua; os números, consulta-se.

32 registradores é pouco

Um vetor com mil números não cabe na bancada. Ele fica no almoxarifado — e é preciso buscá-lo:

```
lw  $t0, 8($s0)
```



É formato I. O imediato, que hoje foi um número para somar, vira um pedaço de endereço.

Na aula 3: a memória.